中山大學
SUN YAT-SEN UNIVERSITY

Lecture 3
Stacks



Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

- Stack Specifications
- Implementation of Stacks
- Application: Bracket Matching
- Application: A Desk Calculator
- Application: Translation
- Abstract Stacks

Stack

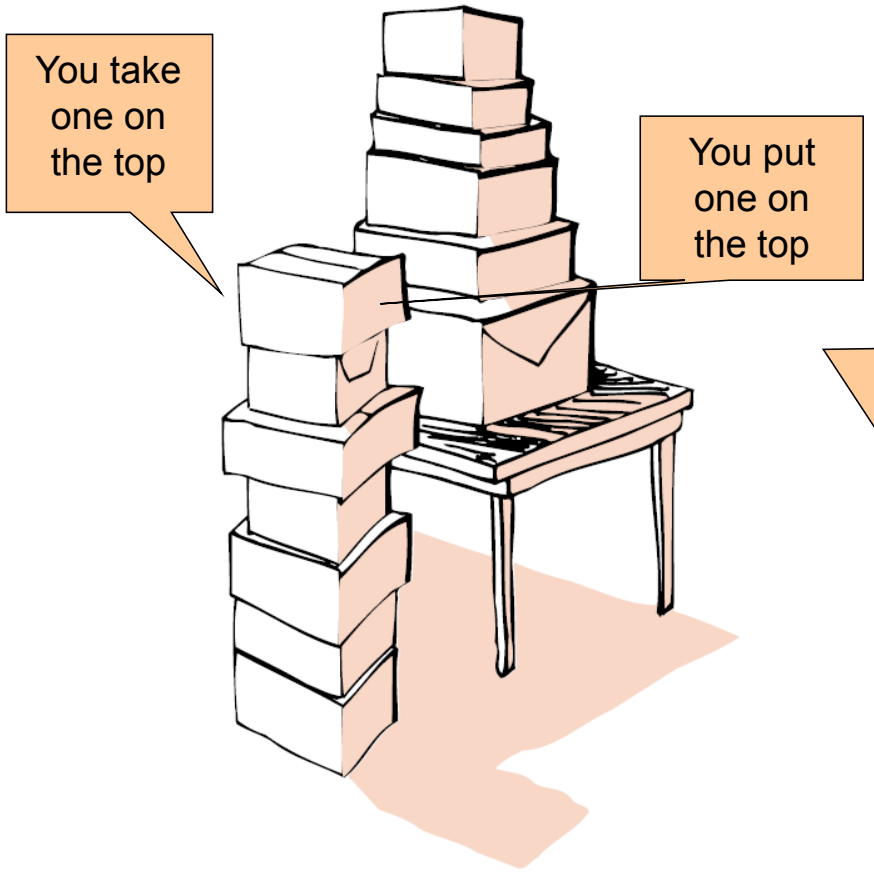
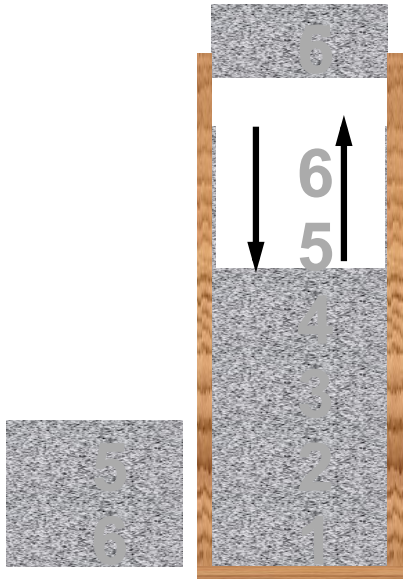


Image that you want to have a program that reads a list of integers and then output them in the reverse order.

How do you program it?

Definition

- A stack is a data structure in which all insertions and deletions of entries are made at one end, called the top of the stack. The last entry which is inserted is the first one that will be removed. The operations perform in a Last-In-First-Out (LIFO) manner.



Stacks in STL

- The standard template library (abbreviated **STL**) in C++ contains much useful information, many functions, and many classes.

- We can include the STL stack implementation into our programs with the directive

```
#include <stack>
```

and then we can define initially empty stack objects and apply methods called push, pop, top, and empty.

Stacks in STL

- In C++, a template construction allows us to create data structures whose entries have different types.

stack<double> numbers or **stack<int> numbers**

- The first template parameter specifies what sort of items the stack contains. It has a second template parameter that has a default value. The second parameter controls what sort of stack implementation will be used.

```
template < class T, class Container = deque<T> > class stack;
```

使用 deque 作为底层容器。

```
// 使用默认的 deque 作为底层容器
std::stack<int> s1;
s1.push(1);
s1.push(2);
s1.push(3);

while (!s1.empty()) {
    std::cout << s1.top() << " ";
    s1.pop();
}
std::cout << std::endl;

// 使用 vector 作为底层容器
std::stack<int, std::vector<int>> s2;
s2.push(4);
s2.push(5);
s2.push(6);

while (!s2.empty()) {
    std::cout << s2.top() << " ";
    s2.pop();
}
std::cout << std::endl;
```

3 2 1

6 5 4

Example: Reversing a List

```
#include <stack>

int main( )
/* Pre:   The user supplies an integer n and n decimal numbers.
   Post:  The numbers are printed in reverse order.
   Uses:  The STL class stack and its methods  */
{
   int n;
   double item;
   stack<double> numbers;        //   declares and initializes a stack of numbers

   cout  << " Type in an integer n followed by n decimal numbers."
         << endl
         << " The numbers will be printed in reverse order."
         << endl;
   cin   >> n;

   for (int i = 0;  i < n;  i++) {
      cin >> item;
      numbers.push(item);
   }

   cout << endl << endl;

   while (!numbers.empty( )) {
      cout << numbers.top( ) << " ";
      numbers.pop( );
   }
   cout << endl;
}
```

Implementation of Stacks

- 用Stack_entry表示栈的元素的类型(如用typedef 说明它所指的类型)；

  typedef int Stack_entry; or, **typedef char Stack_entry;**

- 或者使用模板类表示栈的元素的类型

- 使用枚举类型Error_code表示错误类型；

  enum Error_code { success, underflow, overflow};

- Exception handling: methods should detect errors and let the client to decide what to do.

Specification for Methods

Error_code Stack :: pop( );

Pre: None.

Post: If the Stack is not empty, the top of the Stack is removed. If the Stack is empty, an Error_code of underflow is returned and the Stack is left unchanged.

const: item cannot be changed

Error_code Stack :: push(**const** Stack_entry &item);

Pre: None.

Post: If the Stack is not full, item is added to the top of the Stack. If the Stack is full, an Error_code of overflow is returned and the Stack

Reference: no need to copy a possible big data.

Specification for Methods

```
Error_code Stack :: top(Stack_entry &item) const;
```

Pre: None.

Post: The top of a nonempty Stack is copied to item. A code of fail is returned if the Stack is empty.

const: method cannot change anything.

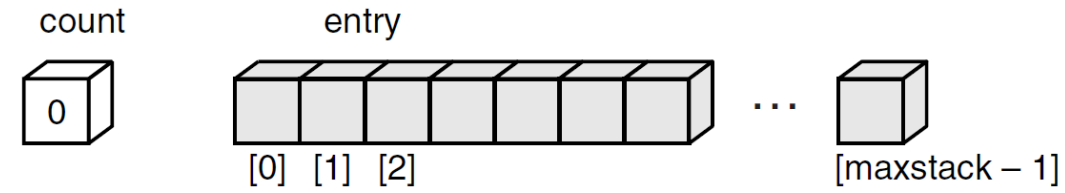
```
bool Stack :: empty( ) const;
```

Pre: None.

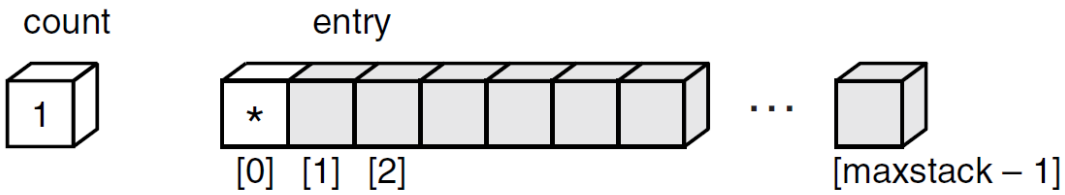
Post: A result of **true** is returned if the Stack is empty, otherwise **false** is returned.

Implementing a Contiguous Stack

- We set up an array to hold the entries in the stack and a counter to indicate how many entries there are.



(a) Stack is empty



(b) Push the first entry.

Implementing a Contiguous Stack

```
const int maxstack = 10;          //  small value for testing

class Stack {
public:
   Stack( );
   bool empty( ) const;
   Error_code pop( );
   Error_code top(Stack_entry &item) const;
   Error_code push(const Stack_entry &item);

private:
   int count;
   Stack_entry entry[maxstack];
};
```

What if?
void pop();
Stack_entry top() const;
void push(const Stack_entry &);

Storage structure

- 如何在计算机内表示栈的元素或者栈的存储结构呢?
- 栈的存储表示应该
  - 表现元素之间的关系;
  - 利于栈的运算的实现。
- 高级语言提供两种基本表示方法:
  - 使用数组的连续结构;
  - 使用指针的链式结构。



Application: Bracket Matching

- **Problem:**
  - We develop a program to check that brackets are correctly matched in an input text file. We limit our attention to the brackets {, }, (,), [, and].
- **Algorithm:**
  - Read the file character by character
  - Each opening bracket (, [, or { that is encountered is considered as unmatched and is stored until a matching bracket can be found. Any closing bracket),], or } must correspond, in bracket style, to the last unmatched opening bracket, which should now be retrieved and removed from storage.
  - Finally, at the end of the program, we must check that no unmatched opening brackets are left over.

Application: Bracket Matching

- 问题：检查程序文件中的括号是否匹配。括号包括：{,},(,),[,].
- 算法思想：
  - 逐个读入字符，忽略非括号字符；
  - 如果是左括号，记下来；
  - 如果是右括号，检查是否和最近读入的左括号匹配；如果不匹配，则匹配失败；
  - 重复上述过程，如果读到文件尾部时，没有未匹配的左括号留下，则匹配成功，否则，匹配失败。

Application: Bracket Matching

- 数据结构的选择：需要记录未匹配的左括号；未匹配的左括号是按照后记录下的先匹配，故应该使用栈。
- Implementation：
  - Use stack to store unmatched left br
  - If a left bracket is read, push in st
  - If a right bracket is read, check if it matches the top of the stack. Pop off the top if it matches. Report bad match if not.
  - In the end the stack should be empty if matching is correct.



Application: Bracket Matching

```
int main( )
/* Post:  The program has notified the user of any bracket mismatch in the standard input
          file.
   Uses:  The class Stack.  */
{  Stack openings;
   char symbol;
   bool is_matched = true;
   while (is_matched && (symbol = cin.get( )) != '\n') {
      if (symbol == '{' || symbol == '(' || symbol == '[')
         openings.push(symbol);
      if (symbol == '}' || symbol == ')' || symbol == ']') {
         if (openings.empty( )) {
            cout << "Unmatched closing bracket " << symbol
                 << " detected." << endl;
            is_matched = false;
         }
         else {
            char match;
            openings.top(match);
            openings.pop( );
            is_matched = (symbol == '}' && match == '{')
                         || (symbol == ')' && match == '(')
                         || (symbol == ']' && match == '[');
            if (!is_matched)
               cout << "Bad match " << match << symbol << endl;
         }
      }
   }
   if (!openings.empty( ))
      cout << "Unmatched opening bracket(s) detected." << endl;
}
```



Application: A Desk Calculator

- 算术表达式的三种表示形式

  1.前缀表达式(Polish notation)

  2.中缀表达式(for person use)

  3.后缀表达式(Reverse Polish Notation)

- 三种表达式之间的转换：按运算的优先次序全部加上括号，逐个括号写成另一种表示式

  (括号——> * ， /——> +， -)

- 对比:

  1.+*a-bcd 前缀

  2.(a*(b-c))+d 中缀

  3.abc-*d+ 后缀

- 波兰记号(The Polish Notation)的好处: 不需要括号

Application: A Desk Calculator

- Exercise: Use Polish notation and Reverse Polish notation to represent the following expressions:
- ((1+2)*(3+4)-5)/6
- 1+2+3+4+5
- 1+(2+(3+(4+5)))

- *If you have access to a UNIX system, you can experiment with a reverse Polish calculator with the command* **dc**.

Application: A Desk Calculator

- In a Reverse Polish calculator, the operands (numbers, usually) are entered before an operation (like addition, subtraction, multiplication, or division) is specified. The operands are pushed onto a stack. When an operation is performed, it pops its operands from the stack and pushes its result back onto the stack. Take '+' as an example:

```
case '+':
   if (numbers.top(p) ==  underflow) cout << "Stack empty" << endl;
   else {
      numbers.pop( );
      if (numbers.top(q) ==  underflow) {
         cout << "Stack has just one entry" << er
         numbers.push(p);
      }
      else {
         numbers.pop( );
         if (numbers.push(q + p) ==  overflow)
            cout << "Warning: Stack full, lost resu
      }
   } break;
```

步骤：
1. **反转输入的中缀表达式**：将中缀表达式的所有操作数和运算符反转。
2. **转换为后缀表达式**：使用中缀转后缀的算法（也称为逆波兰表达式）。
3. **反转结果**：将得到的后缀表达式反转，得到前缀表达式。

详细步骤：

1. 反转输入的中缀表达式
例如，中缀表达式 (A + B) * (C - D) 反转后变为)D - C(*)B + A(。

2. 转换为后缀表达式
使用中缀转后缀的算法：
- 创建一个空栈和一个空输出列表。
- 从左到右扫描反转后的表达式。
- 遇到操作数时，直接添加到输出列表。
- 遇到运算符时，如果栈顶的运算符优先级高于或等于当前运算符，则弹出栈顶运算符并添加到输出列表，直到栈为空或栈顶运算符优先级低于当前运算符。
- 遇到左括号时，直接压入栈。
- 遇到右括号时，弹出栈中的运算符并添加到输出列表，直到遇到左括号。
- 扫描结束后，将栈中剩余的运算符全部弹出并添加到输出列表。

对于反转后的表达式)D - C(*)B + A(：
- 输出列表：D C - B A + *

3. 反转结果
将后缀表达式 D C - B A + * 反转，得到前缀表达式 * + A B - C D。

最终结果
中缀表达式 (A + B) * (C - D) 转换为前缀表达式为 * + A B - C D。

Application: Translation

- 中缀表达式a + b*c + (d * e + f) * g，其转换成后缀表达式则为a b c * + d e * f + g * +。
- 转换过程需要用到栈，具体过程如下：

1）如果遇到操作数，我们就直接将其输出。

2）如果遇到操作符，则我们将其放入到栈中，遇到左括号时我们也将其放入栈中。

3）如果遇到一个右括号，则将栈元素弹出，将弹出的操作符输出直到遇到左括号为止。**注意，左括号只弹出并不输出。**

4）如果遇到任何其他的操作符，如（“+”，“*”，“（”）等，**从栈中弹出元素直到遇到发现更低优先级的元素(或者栈为空)为止（即高可压低）**。弹出完这些元素后，才将遇到的操作符压入到栈中。**有一点需要注意，只有在遇到") "的情况下我们才弹出" ("，其他情况我们都不会弹出" ("。**

5）如果我们得到了输入的末尾，则将栈出剩余的元素依次弹出

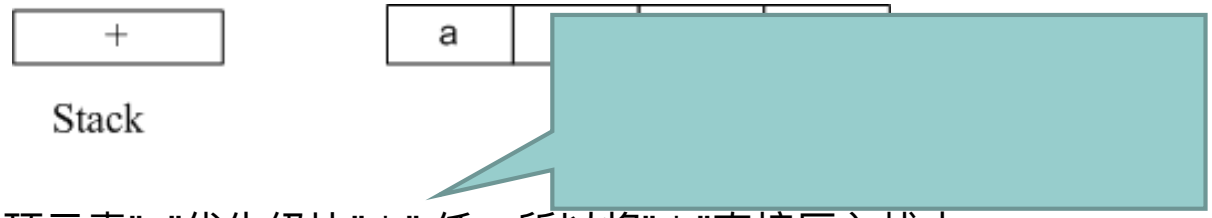
Translation

1）首先读到a，直接输出。

2）读到“+”，将其放入到栈中。

3）读到b，直接输出。

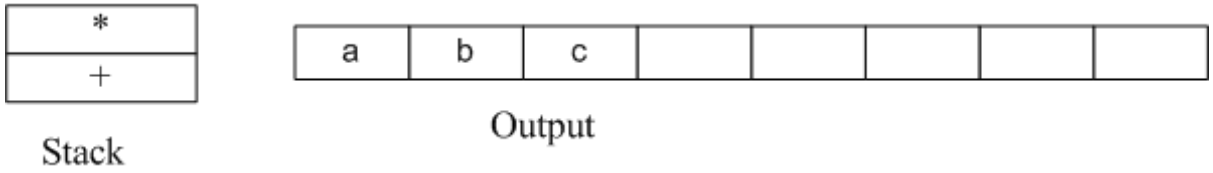
此时栈和输出的情况如下：



4）读到“*”，因为栈顶元素"+"优先级比" * " 低，所以将" * "直接压入栈中。

5）读到c，直接输出。

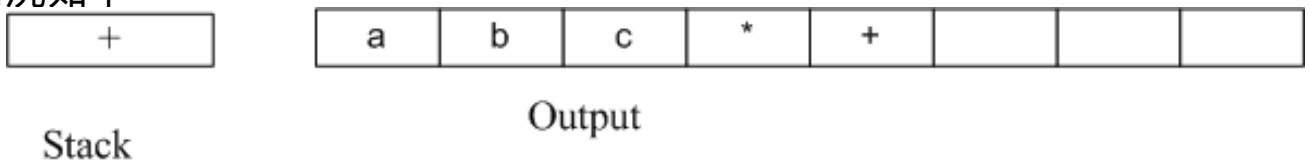
此时栈和输出情况如下：



Translation

6）读到" + "，因为栈顶元素" * "的优先级比它高，所以弹出" * "并输出， 同理，栈中下一个元素" + "优先级与读到的操作符" + "一样，所以也要弹出并输出。然后再将读到的" + "压入栈中。

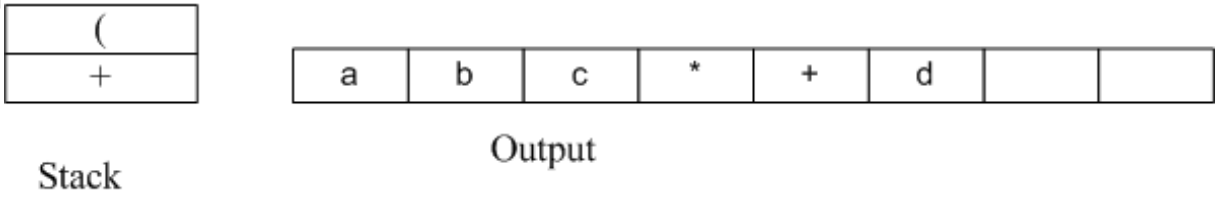
此时栈和输出情况如下：



7）下一个读到的为"("，它优先级最高，所以直接放入到栈中。

8）读到d，将其直接输出。

此时栈和输出情况如下：

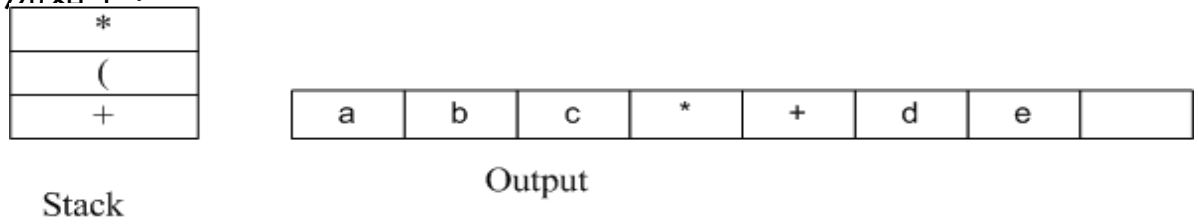


Translation

9）读到" * "，由于只有遇到") "的时候左括号"("才会弹出，所以" * "直接压入栈中。

10）读到e，直接输出。

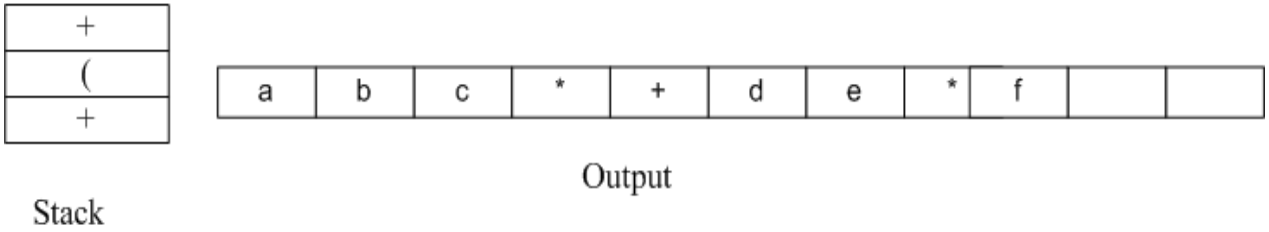
此时栈和输出情况如下：



11）读到" + "，弹出" * "并输出，然后将"+"压入栈中。

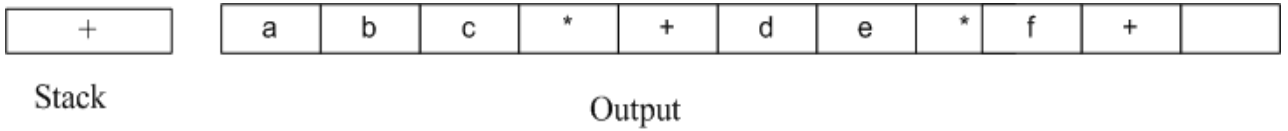
12）读到f，直接输出。

此时栈和输出情况：

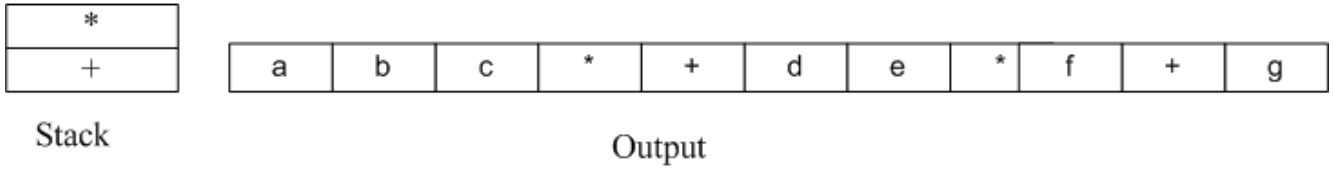


Translation

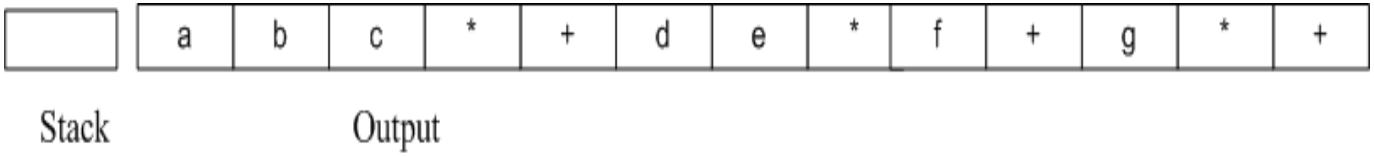
13）接下来读到“）”，则直接将栈中元素弹出并输出直到遇到"("为止。这里右括号前只有一个操作符"+"被弹出并输出。



14）读到" * "，压入栈中。读到g，直接输出。



15）此时输入数据已经读到末尾，栈中还有两个操作符“*”和" + "，直接弹出并输出。



Abstract Stacks

DEFINITION A ***stack*** of elements of type T is a finite sequence of elements of T, together with the following operations:

1. *Create* the stack, leaving it empty.
2. Test whether the stack is *Empty*.
3. *Push* a new entry onto the top of the stack, provided the stack is not full.
4. *Pop* the entry off the top of the stack, provided the stack is not empty.
5. Retrieve the *Top* entry from the stack, provided the stack is not empty.

Thank you!

